

## STM32G4 Programming Requirements

22 May 2026

Author: Everett P

The HVPS V3 Controller (mislabelled V32 on the PCB silkscreen) is designed around the STM32G474QET6 MCU. This SKU was selected due to its available advanced timer and high-resolution timer functionalities which will be discussed throughout this document.

*Note: In this document, I use the term “PWM” (pulse width modulation) to refer to both pulse **width** and pulse **frequency** modulation. Due to the flexibilities required of the HVPS controller, and the nature of embedded systems timer programming, the distinction between these is often blurry.*

The primary function of the HVPS V3 is to output 3 channels of phases (phases U, V, W) and their complementary phases (phases UN, VN, WN) of PWM with deadtime insertion via fiber optic transmitters, which are soldered to the PCB and accessible for use controlling switching power supplies at WHAM. There are two primary types of PWM output required.

- 1. Pulse-frequency modulation.** For this mode, the duty cycle of each output waveform is held constant throughout the PWM “shot” at approximately 50%. For the duration of the controller PWM output, the frequency of the signal may change as quickly as a per-period basis, but no faster. All three phases are programmed for the same output, but out of phase by 120degrees. Phase U with 0deg phase shift, Phase V with 120deg phase shift, and Phase W with -120deg (240deg) phase shift.
- 2. Pulse-width modulation.** For this mode, the PWM frequency is held constant throughout the shot. Duty cycles must be adjustable on a per-period basis. Each phase must have independently programmable duty cycles, allowing unique PWM output signals from each phase. However, each phase will have the same frequency.

When not actively outputting PWM signals, all output channel pins should be held LOW. A watchdog timer TIM6 is programmed in single-pulse mode with a period defined by the user, between 0ms and 100ms (lower bound of 0.1ms or sufficiently small is OK). Once TIM6 period elapsed callback is called, halt all PWM output even if more values remain in the register.

The required output frequency ranges are 5kHz to 200kHz with 100Hz steps (500Hz steps OK), and duty cycles between 5% and 95% with granularity no coarser than 1% for duty cycle.

**Problem statement/sketch:**

Design and implement a lightweight and portable library of functions (`hrtim_pwm.c/.h`) for the STM32G474QET6. This library should include at minimum:

1. Variables:
  - a. `int array_index` to store period and compare array index variables. One variable will be sufficient since all arrays point to corresponding periods.
  - b. `float pulse_duration_ms` default 5ms. User-input shot length value that determines how long of a period TIM6 has. Range 0ms to 100ms.
  - c. 4 arrays: `PER[n]`, `CMP_U[n]`, `CMP_V[n]`, `CMP_W[n]` for storage and indexing of shared period register arrays, and respective compare register arrays for U, V, W phases.
2. Initialization functions: pre-ARM state configuration of HRTIM and output channels.
3. Public function `PWM_ARM()` : Preload all HRTIM shadow registers with index-0 values, apply phase offsets to counter preloads.
  - i. Safety check
  - ii. Cleanly initialize HRTIM units and force outputs LOW.
  - iii. Configure deadtime
  - iv. Enable preload of period and compare register arrays
  - v. Load `array_index=0` period and compare register values to timer units
4. Public function `PWM_FIRE()` : Simultaneously release all three HRTIM timer units and start the shot-duration watchdog timer (TIM6). Outputs begin switching immediately.
  - i. Un-force outputs from LOW state. HRTIM now controls timer outputs
  - ii. Enable Timer A period interrupt. Timer A will act as a master reference whose ISR will pre-load the next period's `PER` and `CMP_x` register values for each channel, advance the `array_index` counter.
  - iii. Start all 3 timers (A, B, E?) simultaneously
  - iv. Start shot duration timer TIM6 -- or other general purpose timer. Configure that timer's PSC and ARR such that `ARR = pulse_duration_ms * counts_per_second` and set this timer to one pulse mode. When this timer's ISR runs, it should call `PWM_STOP()`. Once this timer is configured, let it sit idle, not running. It will start with an interrupt.
5. Public function `PWM_STOP()` : Disable shot-duration timer TIM6. Force all output channel outputs LOW. Disable timer units A, B, E(?). Disable Timer A period interrupt. Clear all timer interrupt flags, including shot duration timer flags. Reset `array_index = 0`.
6. Interrupt handler `HRTIM_TIMA_IRQHandler()` : running up to 200kHz, so this function must be efficient.
  - i. Confirm the interrupt was due to period update event on timer A.
  - ii. Clear interrupt flag

## STM32G4 Programming Requirements

22 May 2026

Author: Everett P

- iii. **Bounds check** -- if we have exceeded the length of `PER[n]` array, call `PWM_STOP()`
  - iv. **Advance index:** `array_index = array_index + 1;`
  - v. **Write next values into shadow register:**
    - `TimerA.PER = PER[array_index]`
    - `TimerA.CMP = CMP_U[array_index]`
    - `TimerB.PER = PER[array_index]`
    - `TimerB.CMP = CMP_V[array_index]`
    - `TimerE.PER = PER[array_index]`
    - `TimerE.CMP = CMP_W[array_index]`
7. Interrupt handler `ShotDuration_TIM_IRQHandler()` : Clear shot-duration timer interrupt flag, call `HRTIM_PWM_Stop()`

### Useful file references:

- [HVPS\\_V3\\_PDRz\\_260226.pdf](#): brief overview of HVPS V3 PCB design
- [KiCAD project HVPS\\_Controller\\_V3](#): project schematic and layout for physical pinmappings
- [STM32G4 Pin Mapping.xlsx](#): pin mappings for this project